



Fallstudie: Erarbeitungsphase

Internationale Hochschule

Duales Studium

Studiengang: Informatik

Technical Softwaredesign

Jana Meyer: 10236813

Lena Pannbacker: 1240698

Dozent: Christoph Schöning

Abgabedatum: 31.09.2026

Phase 2: Erarbeitungsphase

In der Analyse wurden 23 Probleme im Code identifiziert, zehn davon wurden für das Refactoring genauer betrachtet. Die Anwendung besitzt Klassen, aber kaum Objektorientiertes Design. TaskManager vereint sieben Verantwortlichkeiten: God Object, SRP-Verletzung, Database mischt Persistenz mit Domänenvalidierung und verwaltet zwei Aggregate zugleich, Benachrichtigungen laufen über duplizierte if/elif-Kaskaden (notify/notify_urgent) und es existieren keinerlei Abstraktionen. Zusätzlich erzeugt jede Klasse ihre Abhängigkeiten selbst, wodurch mehrere Database-Instanzen mit inkonsistentem Zustand entstehen. Auf dieser Basis wurde das Zielbild als ADR-001 Hexagonale Architektur festgehalten, sowie ADR-002 Taskmanager in Services aufteilen und in drei Refactorings überführt.

1. Betrachtete Probleme:

#	Problem / Code Smell	Einordnung	Hauptfolge	Refactoring
1	TaskManager = God Object	SRP verletzt, geringe Kohäsion	Zu viele Verantwortlichkeiten, hohe Kopplung	In TaskService, TaskRepository, NotificationService, Reminder-Service aufteilen
2	TaskManager hängt direkt an konkreten Klassen (Database, UserManager, ...)	DIP verletzt	Schwer test- und austauschbar	Dependency Injection + Interfaces/Protokolle
3	Database mischt Persistenz & Validierung	SRP verletzt, Schichtverletzung	Geschäftslogik in Infrastruktur	Repositorys nur für Persistenz; Validierung in Service/Domäne
4	NotificationCenter mit großer if/elif-Kette	OCP verletzt	Neue Kanäle erfordern Codeänderungen	Strategy Pattern / Polymorphie + Registry
5	EmailService verschickt auch SMS & Push	SRP verletzt, irreführender Name	Geringe Kohäsion	EmailSender, SmsSender, PushSender
6	UserManager übernimmt zu viele Aufgaben	SRP verletzt	Service, Repository & Factory vermischt	UserRepository, UserService, UserFactory trennen

7	User-Unterklassen verhalten sich unterschiedlich	LSP verletzt	Inkonsistente API, schlechte Testbarkeit	Einheitliche Verträge oder Rollen/Policies statt Vererbung
8	Keine klaren Interfaces	ISP verletzt	Unnötige Abhängigkeiten, schlechte Austauschbarkeit	Kleine, gezielte Interfaces; z. B. Notifier
9	Jeder Speichervorgang überschreibt beide JSON-Dateien	Architektur-/Skalierungsproblem	Ineffizient, schlecht skalierbar	Repository-Interface + getrennte Task-/User-Speicher
10	Magic Numbers (1, 2, 3, 4)	Code Smell	Schlechte Lesbarkeit & Wartbarkeit	Enum oder benannte Konstanten (Priority.HIGH, ...)

2. ADRs

ADR-001: Hexagonales Architekturmodell

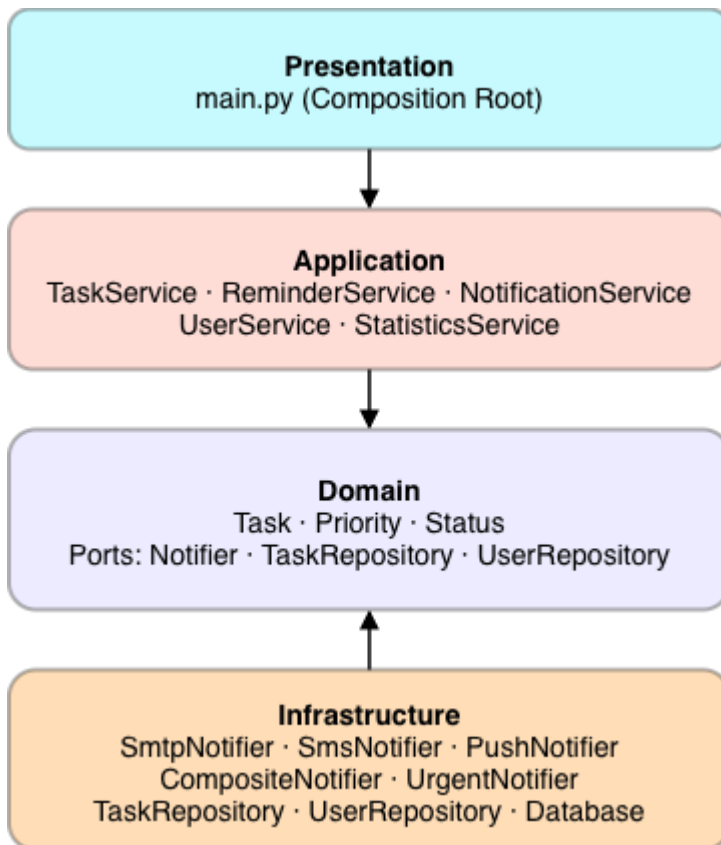
Kontext:

- Die in den Datei-Headern behaupteten Schichten werden nicht eingehalten
- Es gibt keine Abstraktionen
- Es entstehen mehrere Database Instanzen mit inkonsistenten Zuständen

Entscheidung:

- Domain: Task, User, Priority, Status; Ports als TaskRepository, UserRepository, Notifier
- Application: TaskService, TaskRepository, UserRepository bekommen ihre Ports per Konstruktor injiziert
- Infrastructure: Adapter: JsonTaskRepository, SmtplibNotifier
- Main.py: Composition Root: Hier werden die Klassen erzeugt und verdrahtet

Schichtendiagramm:



Alternativen:

- MVC: TaskMaster hat keine View --> verworfen
- Schichten: ohne Ports bleibt die Abhängigkeit Application – Infrastructure bestehen --> verworfen

Konsequenzen:

- + Kernlogik testbar
- + Ein Änderungsgrund pro Klasse, Architektur im Code sichtbar
- Mehr Dateien, höhere Komplexität
- Für diese Projektgröße ist der Aufwand grenzwertig

ADR-002: TaskManager in Services aufteilen

Kontext:

- TaskManager hat sieben Verantwortlichkeiten: Erstellung, Validierung, Persistenz, Nutzerzugriff, Benachrichtigung, Reminder und Formatierung
- Ist nicht isoliert testbar
- God-Objekt

Entscheidung:

Aufteilung nach Verantwortlichkeit:

Einheit	Schicht	Verantwortung
Task	Domain	Regeln, Selbstvalidierung
TaskService	Application	Fachliche Task-Operationen
TaskRepository	Infrastructure	Speichern/Laden von Tasks
NotificationService	Infrastructure	Benachrichtigungen
ReminderService	Application	Erinnerungslogik

Alternativen:

- Alles in TaskMaster lassen, nur Methoden besser ordnen: behebt nicht das Problem mit der Testbarkeit und der Kopplung --> verworfen
- Nur Persistenz herauslösen: löst SRP nur zur Hälfte, Benachrichtigung und Reminder bleiben vermischt --> verworfen

Konsequenzen:

- + Jede Klasse einzeln testbar, ein Änderungsgrund pro Klasse
- + self.cnt und self.email entfallen
- Mehr Klassen und mehr Dateien

3. Refactoring Umsetzungsfälle

Im folgenden Abschnitt werden die drei vorgenommenen Refactoring-Vorschläge erläutert

1. Refactoring-Vorschlag

- Behandelt Problem 1
- TaskManager aufteilen in z. B.:
 - TaskService → fachliche Task-Operationen
 - TaskRepository → Speichern/Laden von Tasks
 - NotificationService → Benachrichtigungen
 - ReminderService → Erinnerungslogik

2. Refactoring-Vorschlag

- Behandelt Problem 4, 5 & 8
- Notifier-Port erstellen:

- Strategy Pattern

3. Refactoring-Vorschlag

- Behandelt Problem 2
- Composition Root + Konstruktor-Injektion
 - Erzeugung der Objekte nur noch in main
 - Alle anderen Klassen bekommen Abhängigkeiten als Konstruktorparameter und kennen nur die Ports

4. Aktueller Stand:

Im folgenden Abschnitt wird der aktuelle Stand der Erarbeitungsphase aufgezeigt:

- 4.1. Aufteilung des God Objekts: TaskManager wurde zerlegt in TaskService, ReminderService und NotificationService; Persistenz liegt in TaskRepository und UserRepository
- 4.2. Notifier-Port mit Entwurfsmustern: EmailService und NotificationCenter werden durch einen schmalen Notifier-Port ersetzt, implementiert per Strategy (SmtplibNotifier, SmsNotifier, PushNotifier), Composite (CompositeNotifier) und Decorator. (UrgentNotifier)
- 4.3. Konstruktor-Injektion: Services erhalten ihre Abhängigkeiten über den Konstruktor und kennen nur die Ports, die Objekterzeugung wird im Composition Root gebündelt

5. Offene Aufgaben:

Die folgenden Aufgaben werden im Rahmen der Finalisierungsphase behandelt

- Composition Root in main.py vollständig mit den neuen Services verdrahten
- Neues Feature einbinden
- Unit Tests entwickeln
- Test der Kernlogik mit gemockter Infrastruktur